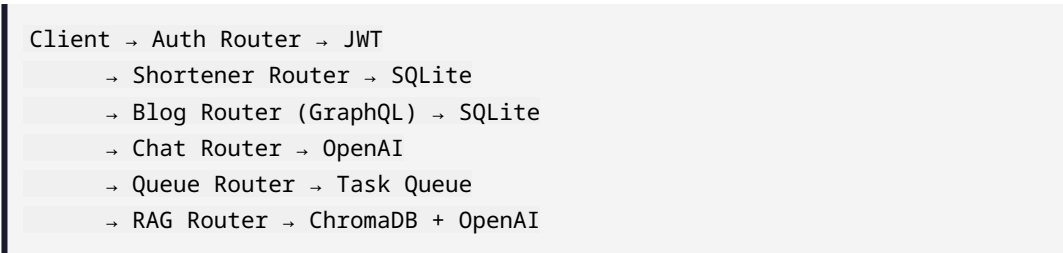


# Python Portfolio — Data Flow Analysis

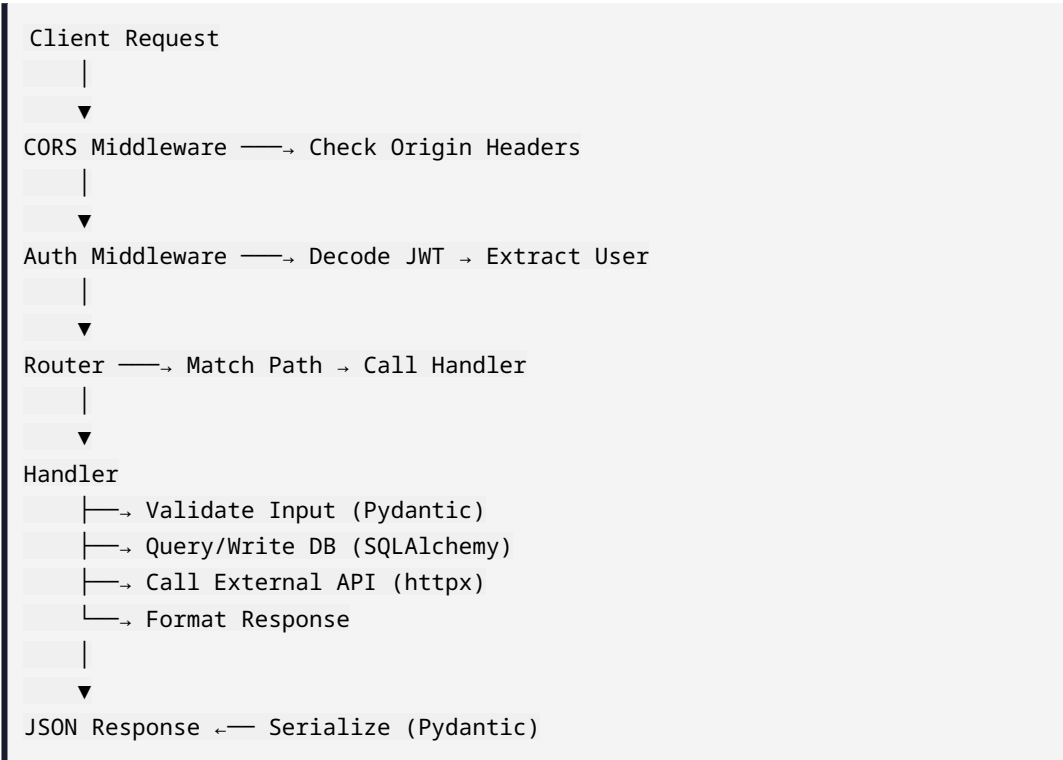
**Project:** Python Portfolio  
**Technology:** Python, FastAPI, SQLAlchemy, SQLite, Pydantic, JWT  
**Location:** /home/dominic/Documents/python-portfolio  
**Report #:** 15/20

## Request Flow



## Data Flow Analysis

### Request Data Flow



## Database Transaction Flow

```
graph TD
    A[Begin Transaction] --> B[Query]
    B --> C[Validate]
    C --> D[Mutate]
    D --> E[Commit]
    E --> F[Rollback]
```

Begin Transaction

Query —→ SELECT with filters

Validate —→ Check business rules

Mutate —→ INSERT/UPDATE/DELETE

Commit —→ On success

Rollback —→ On error

## WebSocket Data Flow (Queue Module)

```
graph TD
    A[Client opens WebSocket] --> B[Server accepts connection]
    B --> C[Task created]
    C --> D[Client receives]
    D --> E[Task completes]
    E --> F[WebSocket closes]
```

Client opens WebSocket

Server accepts connection

Task created → Server pushes progress updates

Client receives: {"progress": 45, "status": "running"}

Task completes → {"progress": 100, "status": "completed"}

WebSocket closes

## External API Data Flow (AI Chat Proxy)

```
graph TD
    A[Client] --> B[Proxy]
    B --> C[OpenAI]
    C --> D[Proxy]
```

Client → POST /v1/chat/completions

Proxy → Add API key → Forward to api.openai.com

OpenAI → SSE stream (tokens)

Proxy → Forward stream as-is



Client → Parse SSE → Render tokens incrementally

## Data Transformations

1. **URL shortening:** Long URL → short\_code (base62 hash)
2. **Auth:** Password → bcrypt hash → DB
3. **RAG:** PDF → Text → Chunks → TF-IDF vectors
4. **Queue:** Task request → DB row → WebSocket progress
5. **Chat:** Message → JSON → API call → SSE stream